unGhost

Deployment Setup Guide

Zero-to-production setup, step-by-step

Audience: Engineer with code-ready repo, no ops setup yet.
Region: India (mumbai)
Stack: Next.js 14 · MongoDB · Cloudflare Containers
Generated: 2026-05-17

Table of contents

1. Pre-flight checklist

Before you start, gather the following. Several of these have wait times measured in days (DLT registration, PhonePe agreement signing), so collect them up front. Skipping this checklist means stalling out three days into setup waiting on a callback.

What you need on hand

- A laptop with a terminal (macOS, Linux, or WSL on Windows). Mac is assumed in commands.
- A credit or debit card with international transactions enabled. Several services bill in USD.
- A working email address you control — preferably **founder@unghost.com** created later, or a personal Gmail for now.
- A phone number for OTPs.
- Indian **PAN card** (personal or company). Required for PhonePe + MSG91.
- Indian **GST registration**. Required for PhonePe merchant agreement. If you do not yet have GST, apply at gst.gov.in — typical turnaround 7 working days.
- Indian **current account** (not savings) in the legal entity's name. Required for PhonePe settlement.
- **MOA / Certificate of Incorporation** if you're a registered entity (Pvt Ltd, LLP). Optional for sole proprietorship.
- A Google account — used for Cloudflare login, Google OAuth setup, and one-tap signups.
- About **USD 30-50** in card balance for first-month service costs.
- Patience — DLT (SMS sender registration) takes 5-15 business days. PhonePe live keys take ~5 working days after KYC.

> **Warning.** Do NOT use personal accounts for anything you will hand off later. Create **founder@unghost.com** first (after Resend domain setup) and re-use it for every signup. Mixing personal + company accounts becomes a nightmare at handover.

Time budget

- Day 1: domain, GitHub, Cloudflare, Mongo, Upstash, Anthropic, Sentry — all setup-able in one focused day (~6h).
- Day 2-7: DLT (MSG91 SMS approval) waiting + PhonePe KYC review running in parallel.
- Day 8-10: PhonePe live keys arrive, MSG91 templates approved, real OTPs + payments work end-to-end.
- Day 10-14: Cloudflare Container deploy, GitHub Actions wiring, first production tag pushed.

2. Architecture overview

unGhost is a Next.js 14 app shipped as a Docker container running on Cloudflare Containers. Every external service is hit through an adapter in `server/integrations/` with a mock fallback so dev works offline. In prod, env vars flip every adapter to live.

Request flow

```
Browser
|
v
Cloudflare DNS (unghost.com) --- WAF, DDoS protection
|
v
Cloudflare Container (unghost-prod)
|--- Next.js 14 (server components + API routes)
|
+--> MongoDB Atlas (ap-south-1, Mumbai)
+--> Upstash Redis (Mumbai)
+--> Anthropic Claude API
+--> MSG91 (SMS / OTP)
+--> Resend (email)
+--> PhonePe Payment Gateway
+--> Pusher Channels (websockets)
+--> Inngest (background jobs)
+--> 100ms (live video rooms)
+--> Cloudflare R2 (resume + asset storage)
+--> Sentry (error tracking)
+--> Better Stack (uptime monitoring)
```

Two environments

- **Staging**: `staging.unghost.com`. Auto-deploys on every merge to `main`. Connects to staging Atlas + staging Upstash. Optionally runs PhonePe in sandbox mode.
- **Production**: `www.unghost.com`. Deploys on git tag push. Requires 2-approver gate in GitHub Environments.

What lives where

Application	Cloudflare Container · Docker image · 2 replicas autoscaling
Database	MongoDB Atlas M10 prod, M0 free staging · ap-south-1 (Mumbai)
Cache + OTP	Upstash Redis · Global / Mumbai region
Object storage	Cloudflare R2 · resumes, company logos, bootcamp covers
DNS + CDN + WAF	Cloudflare (free plan adequate to start)
Secrets	Cloudflare Containers env vars + GitHub Actions secrets
Monitoring	Sentry (errors), Better Stack (uptime), Slack (alerts)
CI/CD	GitHub Actions building Docker image and deploying via wrangler

3. Service-by-service setup

Each subsection follows the same pattern: **what** the service is, **why** we use it, **signup steps**, the **env vars** it produces, exactly what to **paste** into `.env.local` and the Cloudflare Container env, how to **verify** it works, and the most common **errors**.

> **Note.** Throughout this section, replace **unghost.com** with your actual domain if you registered a different name. All commands assume your terminal is in the repo root.

3.a Domain registration

What and why

You need a domain to point users at. We recommend Cloudflare Registrar because it integrates seamlessly with Cloudflare DNS (next step) and charges at-cost (no markup). Alternative: Namecheap, GoDaddy, BigRock.

Steps — Cloudflare Registrar

1. Open `https://www.cloudflare.com/products/registrar/` and click **Sign up**.
2. Create a Cloudflare account using **founder@unghost.com** (or your personal Gmail if you haven't set up Resend yet — you can change later).
3. Verify your email by clicking the link Cloudflare sends.
4. Add a payment method: **Dashboard** → **Billing** → **Payment methods** → **Add card**.
5. Go to **Domain Registration** → **Register Domains**.
6. Search for **unghost.com**. If available, click **Purchase**. Cost ≈ USD 10/year for .com.
7. Complete WHOIS info — use your legal name and an address you control. Privacy is on by default.
8. Wait 5-10 minutes for the domain to become active.

Steps — Namecheap (alternative)

1. Open `https://www.namecheap.com`, sign up, verify email.
2. Search and buy **unghost.com**.
3. After purchase, you will switch nameservers to Cloudflare in step 3.c below.

Env vars produced

```
# .env.local + Cloudflare Container env
NEXT_PUBLIC_APP_URL=https://www.unghost.com
NEXTAUTH_URL=https://www.unghost.com
```

Verify

- Visit `https://www.unghost.com` — should show the registrar parking page (until DNS is wired in 3.c).
- Run `whois unghost.com` in terminal — should show your registrar.

Cost gotcha

Auto-renew is on by default at Cloudflare; off by default at Namecheap. Set a calendar reminder one month before expiry. Losing the domain means losing your brand.

3.b GitHub repo + branch protection

What and why

Your code lives here. GitHub Actions is what runs the CI build, smoke tests, and the wrangler deploy command. Branch protection on `main` stops accidental force-pushes.

Steps

1. Push your local repo to a new GitHub repository (private). Name: **unghost**.
2. Open **Repo** → **Settings** → **Branches** → **Add branch protection rule**.
3. Branch name pattern: `main`.
4. Enable: **Require a pull request before merging** (1 approval).
5. Enable: **Require status checks to pass before merging**. Select `verify` and `e2e` once CI has run at least once.
6. Enable: **Require branches to be up to date before merging**.
7. Enable: **Do not allow bypassing the above settings**.
8. Click **Save changes**.
9. Now create **Settings** → **Environments** → **New environment** → **production**. Add yourself + tech lead as **required reviewers** (1 reviewer minimum, 2 recommended).
10. Create another environment **staging** — no reviewers required (auto-deploys).

Env vars produced

None — this configures the CI pipeline only.

Verify

- Try pushing directly to `main` from the command line — GitHub should reject.
- Open **Actions** tab. The next push should kick off the CI workflow.

3.c Cloudflare account + DNS + zone setup

What and why

Cloudflare handles DNS, TLS, DDoS, WAF, and (later) the container runtime. Even if your registrar is not Cloudflare, you point the nameservers here for the security and performance perks.

Steps — if you used Cloudflare Registrar

1. From Cloudflare Dashboard, click **Websites** → **Add a site** → enter `unghost.com`.
2. Select the **Free** plan. Click **Continue**.
3. Cloudflare scans existing DNS records — there will be none. Click **Continue**.
4. Nameservers are pre-pointed because the registrar is Cloudflare. You should see **Active** status within a few minutes.

Steps — if you used a different registrar (e.g. Namecheap)

1. Add the site to Cloudflare as above.
2. Cloudflare shows you two nameservers like `aria.ns.cloudflare.com` and `kirk.ns.cloudflare.com`.
3. In Namecheap: **Domain List** → **Manage** → **Nameservers** → **Custom DNS**. Paste both Cloudflare nameservers.
4. Save. Propagation takes anywhere from 5 minutes to 24 hours.
5. Cloudflare will email you when the zone becomes Active.

Add the DNS records (do not skip)

In Cloudflare Dashboard → **unghost.com** → **DNS** → **Records** → **Add record**. Add these four (some come later in this guide):

CNAME @	unghost-prod..containers.cloudflare.com · Proxied · TTL Auto
CNAME www	unghost-prod..containers.cloudflare.com · Proxied
CNAME staging	unghost-staging..containers.cloudflare.com · Proxied
CNAME status	· DNS only (we set this in 3.o)

Note. You don't have the container CNAMEs yet. Come back to this table after step 4 (Cloudflare Containers deployment) and fill in the actual values. For now, the zone exists and SSL is auto-issued in the background.

SSL/TLS settings

1. Go to **SSL/TLS** → **Overview**. Set **Full (strict)**.
2. Go to **SSL/TLS** → **Edge Certificates** → **Always Use HTTPS** — turn ON.
3. Same page → **HTTP Strict Transport Security (HSTS)** → Enable. Max-age 12 months, include subdomains, no preload until you've tested end-to-end.
4. **Minimum TLS Version** → TLS 1.2.

Env vars produced

None directly. But you'll need **CLOUDFLARE_API_TOKEN** + **CLOUDFLARE_ACCOUNT_ID** later for CI.

Verify

- `dig unghost.com NS` should list two `*.ns.cloudflare.com` servers.
- `curl -I https://unghost.com` should return a Cloudflare-served 522 or 525 (because no container yet — that's expected at this stage).

3.d MongoDB Atlas

What and why

Production database. Atlas is MongoDB's hosted service — backups, metrics, replication, PITR (point-in-time recovery), and a free tier (M0) for staging.

Steps

1. Open `https://www.mongodb.com/cloud/atlas/register`. Sign up with **founder@unghost.com**.
2. Verify your email.
3. When prompted to create a project, name it **unghost**.
4. **Deploy your database** screen → choose **Shared (M0)** for now. We'll upgrade prod after.
5. Provider: **AWS** · Region: **Mumbai (ap-south-1)**.
6. Cluster name: `unghost-staging`. Click **Create**.
7. Add a **Database User** — username `app-rw`, password generated by Atlas (copy + paste into a password manager).
8. Add IP Access entry — for staging only, `0.0.0.0/0` is fine. For prod we narrow this in step 11 below.
9. Once cluster is provisioning, scroll down: **Database** → **Connect** → **Drivers** → **Node.js**. Copy the connection string. It looks like `mongodb+srv://app-rw:<password>@unghost-staging.xxxxx.mongodb.net/?retryWrites=true&w=majority`. Replace `<password>` with the actual password and append `&appName=unghost`. Save it.
10. Repeat all of the above for a second cluster. Cluster name: `unghost-prod`. Choose **Dedicated (M10)**, region Mumbai, MongoDB version 7.0.
11. Important — M10 costs ~USD 60/month. You can skip until launch and run staging until soft-launch.
12. For the prod cluster, create **three** database users: `app-rw` (readWrite to unghost), `migrator` (dbAdmin to unghost — used only by CI for migrations), `readonly` (read to unghost — used by analytics or admins).
13. Enable **Point-in-time Recovery (PITR)**: prod cluster → **Backup** → **Configure** → **On**. 7 day window default.
14. Add IP access for prod — for Cloudflare Containers, allow `0.0.0.0/0` for now (we cannot pin egress IPs from CF Containers easily). For tighter security, look into VPC peering after launch.

Database name

The repo expects a database literally called **unghost**. Atlas does not pre-create it — Mongoose creates it on first write. Connection string just needs to point at the cluster.

Env vars produced

```
# .env.local (dev) — keep your docker-compose Mongo for local dev,
# or use the staging cluster directly. Either works:
MONGODB_URI=mongodb://noghost:noghost@127.0.0.1:27018/noghost?authSource=admin

# Cloudflare Container env (staging)
MONGODB_URI=mongodb+srv://app-rw:&lt;pwd&gt;@unghost-staging.xxxxx.mongodb.net/unghost?retryWrites=true&amp;w=majority&amp;appName=unghost

# Cloudflare Container env (production)
MONGODB_URI=mongodb+srv://app-rw:&lt;pwd&gt;@unghost-prod.xxxxx.mongodb.net/unghost?retryWrites=true&amp;w=majority&amp;appName=unghost

# GitHub Actions secret (production migrations only)
PROD_MIGRATOR_MONGODB_URI=mongodb+srv://migrator:&lt;pwd&gt;@unghost-prod.xxxxx.mongodb.net/unghost?retryWrites=true&amp;w=majority
```

Verify

```
# from your local terminal — replace URI with staging
MONGODB_URI="mongodb+srv://..." npx mongosh "$MONGODB_URI"
# should drop into a Mongo shell. Try: db.runCommand({ ping: 1 })
# expected output: { ok: 1 }
```

Common errors

- **connection refused** → IP allowlist missing your IP. Add it under Network Access.
- **authentication failed** → wrong password, or you forgot to URL-encode special chars (#, @, /, !) — encode using `encodeURIComponent` in the password section.
- **queryTxt ETIMEOUT** → DNS issue on your machine. Try `nslookup unghost-prod.xxx.mongodb.net`.

3.e Upstash Redis

What and why

Used for OTP storage, rate-limit counters, email-verify tokens, and the in-memory cache layer. Upstash is serverless Redis — pay per command, generous free tier (10k commands/day).

Steps

1. Open `https://upstash.com`. Sign up with Google or email.
2. Click **Create Database**.
3. Name: `unghost-staging`. Region: **Mumbai (ap-south-1)**. Type: **Regional**.
4. Eviction policy: **allkeys-lru**. Max memory: 256MB (free tier limit).
5. Click **Create**.
6. On the database page, scroll down to **REST API**. Copy the **UPSTASH_REDIS_REST_URL** and **UPSTASH_REDIS_REST_TOKEN**.
7. Create a second database named `unghost-prod` with the same settings. For prod, upgrade max-memory to **1GB** when traffic grows — costs ~USD 0.20/GB-month + per-command price.

Env vars produced

```
# Staging container env
UPSTASH_REDIS_REST_URL=https://intense-mosquito-12345.upstash.io
UPSTASH_REDIS_REST_TOKEN=AYy3AAIncDExxxxxxxxxxxx

# Production container env — different URL + token from prod DB
```

Verify

```
curl -X POST "$UPSTASH_REDIS_REST_URL/set/test/hello" \
-H "Authorization: Bearer $UPSTASH_REDIS_REST_TOKEN"
# expected: {"result":"OK"}

curl "$UPSTASH_REDIS_REST_URL/get/test" \
-H "Authorization: Bearer $UPSTASH_REDIS_REST_TOKEN"
# expected: {"result":"hello"}
```

Cost gotcha

Free tier resets daily. If you hit 10k commands/day in production, switch to **Pay as you go** — it bills around USD 0.2 per 100k commands. Set a **spend alert** at **Account** → **Usage** → **Budget**.

3.f Anthropic Claude API

What and why

Powers AI Coach, resume parsing, JD parsing, match scoring, gauntlet review, skill-verify grading. Without this, every AI feature degrades to the deterministic mock adapter.

Steps

1. Open `https://console.anthropic.com`. Sign up.
2. Verify your email and phone.
3. Add a payment method: **Settings** → **Billing** → **Add payment method**.
4. Add credits: **Settings** → **Billing** → **Add credits**. USD 20-50 is enough to get started for dev.
5. **Set a spend limit**: **Settings** → **Billing** → **Usage limits** → **Set monthly limit**. Recommended USD 100 to start — you can raise it as traffic grows.
6. **Settings** → **API keys** → **Create key**. Name it `unghost-prod`. Copy the value (starts with `sk-ant-`) — Anthropic only shows it once.
7. Create a separate key `unghost-staging` if you want isolated billing visibility.
8. Apply for **production tier** (usage ramp): **Settings** → **Usage tiers** → **Request upgrade**. Default tier is fine for soft launch.

Env vars produced

```
ANTHROPIC_API_KEY=sk-ant-api03-xxxxxxxxxxxxxxxxxxxx
```

Verify

```
curl https://api.anthropic.com/v1/messages \
-H "x-api-key: $ANTHROPIC_API_KEY" \
-H "anthropic-version: 2023-06-01" \
-H "content-type: application/json" \
-d '{ "model": "claude-3-5-haiku-latest", "max_tokens": 16, "messages": [{"role": "user",
"content": "ping"}] }'
# expected JSON with content[0].text saying something like "pong"
```

Cost gotcha

Claude 3.5 Sonnet is ~USD 3 per million input tokens and USD 15 per million output. A single resume parse costs ~USD 0.01. Without spend caps, a bug in the rate limiter could empty your card in hours — set the monthly limit.

3.g Resend (email)

What and why

Sends verify-email, password reset, plan_activated receipts, support-ticket notifications, etc. Resend is a Postmark-style API with a 3000 emails/month free tier.

Steps

1. Open `https://resend.com`. Sign up.
2. Verify email.
3. **Domains** → **Add Domain**. Enter **unghost.com**.
4. Resend shows you DNS records to add — typically one SPF (TXT), one DKIM (TXT or CNAME), and one DMARC (TXT). Keep this tab open.
5. Open Cloudflare DNS for unghost.com in a new tab. For each Resend DNS record:
6. • Type: TXT or CNAME as Resend shows.
7. • Name: as shown (e.g. `resend._domainkey`).
8. • Value: as shown.
9. • TTL: Auto. Proxy: **DNS only** (no orange cloud).
10. Save each record. Back in Resend, click **Verify DNS records**. Sometimes takes 10-20 minutes for DNS to propagate.
11. Once verified, go to **API Keys** → **Create API Key**. Name `unghost-prod`. Permission **Sending access**. Copy the value (starts with `re_`).
12. **Settings** → **From email**: configure `hello@unghost.com` as your default sender. Optional: also configure `support@unghost.com`.

Env vars produced

```
RESEND_API_KEY=re_xxxxxxxxxxxxxxxxxxx
RESEND_FROM=hello@unghost.com
SUPPORT_INBOX_EMAIL=support@unghost.com
```

Verify

```
curl -X POST https://api.resend.com/emails \
-H "Authorization: Bearer $RESEND_API_KEY" \
-H "Content-Type: application/json" \
-d '{ "from": "hello@unghost.com", "to": "you@gmail.com", "subject": "hi", "text": "test"
}'
# Expected JSON with an `id` field. Inbox should receive within 5 sec.
```

Common errors

- **Domain not verified** → wait 15 min, then click **Verify DNS records** again. Make sure CNAME records have proxy turned OFF in Cloudflare.
- **Email lands in spam** → make sure DMARC and DKIM both show green. Send a few test emails to mail-tester.com for a quality score.

3.h MSG91 (SMS + OTP)

What and why

Sends OTP during signup, SLA breach alerts, plan-expiry reminders. MSG91 is the canonical India SMS gateway — supports DLT (TRAI requirement).

> **Danger.** DLT registration is mandatory in India to send **any** commercial SMS. Without DLT-approved templates, your SMS will silently fail or take 12+ hours. Start this on Day 1 — it takes 5-15 working days.

Steps

1. Open `https://msg91.com`. Sign up.
2. Verify your email and phone.
3. Go through KYC: PAN, company name, GST (if applicable). Submit. Approval is usually same-day.
4. Add prepaid balance: ~INR 500 is plenty to start (about 3000 SMS).
5. **DLT registration**: go to **SMS** → **DLT**. MSG91 has guides for each telecom operator — Vodafone Idea, Jio, Airtel, BSNL.
6. Register as an **Entity** on the DLT portal of one major operator (jio.trueconnect.in or similar). You will need: Entity name, PAN, GST, address, authorised signatory.
7. Once entity is approved, register a **Header** (sender ID — up to 6 chars). Suggest `UNGHST` or `NGHOST`.
8. Register a **Content Template** for OTP. Example: `Your unGhost OTP is {{var}}. Valid for 10 minutes. Do not share. - unGhost`
9. Wait for template approval (5-15 working days). Status visible on DLT portal.
10. Back in MSG91 → **Auth Key** menu — copy the auth key.
11. **Templates** → after DLT approval is reflected in MSG91, your template gets an ID. Copy it.
12. **Sender ID** — copy the approved 6-char sender ID.

Env vars produced

```
MSG91_AUTH_KEY=xxxxxxxxxxxxxxxxxxxxxxxx
MSG91_SENDER_ID=UNGHST
MSG91_OTP_TEMPLATE_ID=xxxxxxxxxxxxxxxxxxx
```

Verify

Use MSG91's web console: **OTP** → **Send Test OTP** to your own phone. The SMS should land within 10 seconds. Once you've confirmed, the env vars above will work from the app.

Common errors

- **Invalid Auth Key** → the DLT template is not yet approved. Wait for MSG91 console to show "Template approved".
- **SMS delays of 1-12 hours** → unapproved template or running through fallback DLT route. Re-check template status.
- **OTP not delivered to DND numbers** → enable transactional (vs promotional) category in template.

3.i PhonePe Business

What and why

Processes Pro (■999/mo) and Premium (■4,999 lifetime) subscriptions, plus recruiter sponsorship payments. PhonePe takes ~1.99% per successful transaction.

> **Danger.** PhonePe live keys take 5-7 working days after KYC. Start this immediately. You can develop against the sandbox in the meantime — sandbox keys are issued same-day.

Steps

1. Open `https://business.phonepe.com`. Click **Get Started**.
2. Pick **Payment Gateway** as the product.
3. Fill the application form: business name, PAN, GST, current account, MOA/CoI (if registered entity), website (`https://www.unghost.com`), expected monthly volume.
4. Upload supporting documents (PAN, GST cert, bank statement showing current account).
5. Sign the merchant agreement (e-sign). PhonePe support contacts you within 24 hours.
6. While waiting for live keys: **request sandbox credentials** by emailing `merchantsupport@phonepe.com` with subject **Sandbox keys for unghost.com**. They usually reply within 4 hours with: merchant ID, salt key, salt index, sandbox base URL.
7. Once live keys arrive (5-7 working days after KYC), you get a different merchant ID, salt key, and base URL.
8. **Callback URL whitelisting**: in the merchant dashboard, set the callback URL to `https://www.unghost.com/api/payments/phonepe/webhook` and the redirect URL pattern to `https://www.unghost.com/api/billing/callback`.

Env vars produced (sandbox)

```
PHONEPE_MERCHANT_ID=PGTESTPAYUAT
PHONEPE_SALT_KEY=099eb0cd-02cf-4e2a-8aca-3e6c6aff0399
PHONEPE_SALT_INDEX=1
PHONEPE_BASE_URL=https://api-preprod.phonepe.com/apis/pg-sandbox
```

Env vars produced (live)

```
PHONEPE_MERCHANT_ID=&lt;your live merchant id&gt;
PHONEPE_SALT_KEY=&lt;your live salt key&gt;
PHONEPE_SALT_INDEX=1
PHONEPE_BASE_URL=https://api.phonepe.com/apis/hermes
```

Verify

After deploying staging with sandbox keys: log in as a student, hit /upgrade?to=pro, click Go Pro, and complete the test payment using PhonePe's sandbox test cards. You should land on /upgrade/success.

Common errors

- **X-VERIFY signature mismatch** (HTTP 401 in webhook) → make sure your salt key, salt index, and the path you sign (`/pg/v1/pay`) all match exactly.
- **Test cards don't work in sandbox** → use PhonePe's documented sandbox test UPI (success@ybl) and test cards. Real cards return decline.
- **Webhook never fires** → callback URL not whitelisted, or container 5xx-ed during processing (PhonePe retries 3 times then gives up).

3.j Pusher Channels

What and why

Real-time WebSocket layer for live chat between students and recruiters, live session presence, and dashboard pings. Generous free tier (200k messages/day).

Steps

1. Open `https://pusher.com`. Sign up.
2. Create a new Channels app. Name: **unghost-prod**. Cluster: **ap2 (Mumbai)**.
3. **App keys** tab — copy the four values: app_id, key, secret, cluster.
4. Create a second app for staging: **unghost-staging**, same cluster.

Env vars produced

```
PUSHER_APP_ID=1234567
PUSHER_KEY=xxxxxxxxxxxxxxx
PUSHER_SECRET=xxxxxxxxxxxxxxx
PUSHER_CLUSTER=ap2
```

Verify

Once deployed, open the Pusher dashboard → **Debug Console**. Trigger a message from the app (e.g. send a chat from /student/messages). The event should appear live.

3.k Inngest

What and why

Schedules background jobs: SLA breach sweep, subscription expiry sweep, async DPDP exports. Free tier covers 50k function runs/month.

Steps

1. Open `https://www.inngest.com`. Sign up with GitHub.
2. Create environment **production**.
3. **Manage** → **Event keys** → **Create**. Copy the value.
4. **Manage** → **Signing keys** → **Reveal**. Copy the value.
5. Repeat for a **staging** environment.

Env vars produced

```
INNGEST_EVENT_KEY=evt_xxxxxxxxxxxxxxxxxx
INNGEST_SIGNING_KEY=signkey-prod-xxxxxxxxxxxxxxx
```

Verify

After deploy, in the Inngest dashboard you should see your app auto-register and list the registered functions. Manually trigger a function from the UI to confirm round-trip.

3.l 100ms (video)

What and why

Live video rooms for instructor coaching sessions and pre-screen interviews. Free tier 10k minutes/month.

Steps

1. Open `https://www.100ms.live`. Sign up.
2. Create an app. Region: **India (Mumbai)**.
3. Create a **Template**: name `coaching`. Roles: **host** (instructor) and **guest** (student). Default video on, recording disabled at first.
4. **Developer** → **Access keys** — copy **Access Key** and **App Secret**.
5. **Templates** → note the Template ID for the room.

Env vars produced

```
HMS_ACCESS_KEY=xxxxxxxxxxx
HMS_APP_SECRET=xxxxxxxxxxxxxxxxxxxxxxx
HMS_TEMPLATE_ID=xxxxxxxxxxx
HMS_REGION=in
```

3.m Cloudflare R2 (object storage)

What and why

Stores uploaded resumes (PDF/DOCX), company logos, bootcamp covers, and async DPDP export bundles. R2 has free egress — huge cost saver vs S3.

Steps

1. From the Cloudflare dashboard, go to **R2** → **Get started**.
2. Enable R2 (free tier: 10GB storage, 1M Class A ops/month).
3. **Create bucket** — name `unghost-prod`. Location **Auto**.
4. Create a second bucket `unghost-staging`.
5. For each bucket, click **Settings** → **Public access** → **R2.dev subdomain** → **Allow Access**. Save the public URL.
6. **Manage R2 API Tokens** → **Create API token**. Permissions: **Object Read & Write**. Specify both buckets. Copy **Access Key ID** and **Secret Access Key**.
7. Copy your Cloudflare **Account ID** from the right sidebar of any Cloudflare page.

Env vars produced

```
R2_ACCOUNT_ID=xxxxxxxxxxxxxxxxxxx
R2_ACCESS_KEY_ID=xxxxxxxxxxxxxxx
R2_SECRET_ACCESS_KEY=xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
R2_BUCKET=unghost-prod
R2_PUBLIC_BASE_URL=https://pub-xxxxxx.r2.dev
```

3.n Sentry

What and why

Catches uncaught exceptions from server and client. Linked to releases via GitHub Actions for blame attribution. Free tier 5k events/month.

Steps

1. Open `https://sentry.io`. Sign up.
2. Create organisation **unghost**.
3. Create project **unghost-server**. Platform: **Node.js** → **Next.js**. Copy the DSN.
4. Create project **unghost-client**. Platform: **Browser** → **React**. Copy the DSN.
5. **Settings** → **Account** → **API** → **Auth Tokens** → **Create New Token**. Scopes: **project:releases, project:read, project:write**. Copy the token — this is `SENTRY_AUTH_TOKEN`.
6. From the project settings page, copy the **Project Slug** (e.g. `unghost-server`) — this is `SENTRY_PROJECT`.
7. From the Organization page, copy the **Organization Slug** — this is `SENTRY_ORG`.

Env vars produced

```
SENTRY_DSN=https://xxxxxxxx@o123.ingest.sentry.io/123
NEXT_PUBLIC_SENTRY_DSN=https://xxxxxxxx@o123.ingest.sentry.io/124
SENTRY_TRACES_SAMPLE_RATE=0.2
NEXT_PUBLIC_SENTRY_TRACES_SAMPLE_RATE=0.1
# CI only (GitHub Actions secrets)
SENTRY_AUTH_TOKEN=sntrys_xxxxxxxx
SENTRY_ORG=unghost
SENTRY_PROJECT=unghost-server
```

Verify

Once deployed, hit any URL with `?_sentry_test=1` in dev, or throw a manual error from `/api/debug`. The event should appear in Sentry within 30 seconds.

3.o Better Stack (uptime + status page)

What and why

Public uptime monitor that pings `/api/health` from multiple geo locations every 30 seconds. Also hosts your public status page at **status.unghost.com**.

Steps

1. Open `https://betterstack.com`. Sign up.
2. **Uptime** → **Create monitor**. URL: `https://www.unghost.com/api/health`. Interval: 30 sec. Expected status code: 200. Expected response body to contain: `"ok":true`.
3. Add monitors for: staging health, MongoDB Atlas (optional — Atlas itself has alerts), Cloudflare Container origin.
4. **Status pages** → **Create**. Name **unGhost Status**. Subdomain (free) or custom domain (paid).
5. If custom domain: in Cloudflare DNS, add CNAME **status** → `statuspages.betterstack.com` (DNS only, no proxy).
6. Link each monitor as a service on the status page.
7. **Incidents** → **Notifications** → **Slack**: paste the `#incidents` webhook URL (we set that up in 3.r).

3.p Google OAuth

What and why

Enables the "Continue with Google" button. Optional, but cuts signup friction by ~40%.

Steps

1. Open `https://console.cloud.google.com`. Sign in with founder@unghost.com.
2. Create a new project named **unghost**.
3. **APIs & Services** → **OAuth consent screen** → **External**. App name: **unGhost**. User support email: founder@unghost.com. Developer contact: same. Logo: upload your favicon @ 120x120px.
4. Scopes: add `.../auth/userinfo.email` and `.../auth/userinfo.profile`.
5. Test users: add your own email (only used while app is in Testing).
6. Submit for verification once you go live (review takes 4-6 weeks for sensitive scopes, instant for the basic ones above).
7. **APIs & Services** → **Credentials** → **Create Credentials** → **OAuth client ID**. Application type **Web application**.
8. Authorized JavaScript origins: `https://www.unghost.com` and `https://staging.unghost.com` and `http://localhost:3000`.
9. Authorized redirect URIs: `https://www.unghost.com/api/auth/callback/google`, `https://staging.unghost.com/api/auth/callback/google`, `http://localhost:3000/api/auth/callback/google`.
10. Click **Create**. Copy the Client ID and Client Secret.

Env vars produced

```
GOOGLE_CLIENT_ID=xxxxxxxxxxxx-xxxxxxxxxxxxxxx.apps.googleusercontent.com
GOOGLE_CLIENT_SECRET=GOCSPX-xxxxxxxxxxxxxxxxxx
```

3.q LinkedIn OAuth

What and why

Enables "Continue with LinkedIn" for recruiters. Optional.

Steps

1. Open `https://www.linkedin.com/developers`. Sign in.
2. **My Apps** → **Create app**. App name: **unGhost**. LinkedIn page: create a company page first if you don't have one. Logo upload.
3. **Products** tab → request access to **Sign In with LinkedIn using OpenID Connect**.
4. **Auth** tab → Authorized redirect URLs: same three as Google but ending in `/api/auth/callback/linkedin`.
5. **Auth** tab → copy Client ID and Client Secret.

Env vars produced

```
LINKEDIN_CLIENT_ID=xxxxxxxxxxx
LINKEDIN_CLIENT_SECRET=xxxxxxxxxxxxxxx
```

3.r Slack alerts

What and why

Centralised inbox for deploy notifications, incident pings, error spikes, and ops chatter. Three channels: **#prod** (deploys), **#incidents** (live tick-tock), **#engineering** (everything else).

Steps

1. Create a Slack workspace at `https://slack.com/create` if you don't have one.
2. Create three channels: **#prod**, **#incidents**, **#engineering**.
3. **Apps** → **Custom Integrations** → **Incoming Webhooks** → **Add to Slack**.
4. For each channel, create a webhook. Copy each URL.

Env vars / secrets produced

```
SLACK_WEBHOOK_PROD=https://hooks.slack.com/services/T.../B.../xxxxx
SLACK_WEBHOOK_INCIDENTS=https://hooks.slack.com/services/T.../B.../xxxxx
SLACK_WEBHOOK_ENGINEERING=https://hooks.slack.com/services/T.../B.../xxxxx
```

Verify

```
curl -X POST $SLACK_WEBHOOK_PROD \
-H 'Content-Type: application/json' \
-d '{ "text": "hello from unghost setup" }'
# message should appear in #prod
```

4. Cloudflare Containers deployment

Cloudflare Containers runs your Docker image at the edge. The `Dockerfile` in the repo produces a Next.js standalone image. We deploy it twice: once as **unghost-staging**, once as **unghost-prod**.

Initial setup

1. Install Wrangler CLI: `npm install -g wrangler`.
2. Authenticate: `wrangler login`. Opens browser, grants CLI access to your Cloudflare account.
3. **Cloudflare Dashboard** → **Workers & Pages** → **Containers**. If you don't see Containers, request beta access from `https://www.cloudflare.com/products/containers/`.
4. **Containers** → **Create**. Image source: **Cloudflare Registry**. Name **unghost-staging**. Replicas: 1-2. Memory: 1 GB. CPU: 1.
5. Health check path: `/api/health`. Expected response: status 200, body contains `"ok":true`.
6. Custom domain: `staging.unghost.com`. Cloudflare auto-creates the CNAME if your zone is here.
7. Repeat for **unghost-prod**. Replicas: 2-4. Memory: 2 GB. Custom domain: `www.unghost.com` and the apex `unghost.com`.
8. For prod, choose deploy strategy **Blue-green**.

Set environment variables

On each container's page → **Settings** → **Environment variables**. Paste the assembled .env you've been building section by section.

Sample staging env (full list — copy your real values into each):

```
NODE_ENV=production
NEXT_PUBLIC_APP_VERSION=staging
NEXT_PUBLIC_APP_URL=https://staging.unghost.com
NEXTAUTH_URL=https://staging.unghost.com
NEXTAUTH_SECRET=&lt;openssl rand -base64 32&gt;
DEPLOY_REGION=ap-south-1
LOG_LEVEL=info
CRON_SECRET=&lt;openssl rand -hex 32&gt;

MONGODB_URI=&lt;staging Atlas SRV URI&gt;
UPSTASH_REDIS_REST_URL=&lt;staging Upstash URL&gt;
UPSTASH_REDIS_REST_TOKEN=&lt;staging Upstash token&gt;
ANTHROPIC_API_KEY=&lt;your Anthropic key&gt;

MSG91_AUTH_KEY=&lt;your MSG91 key&gt;
MSG91_SENDER_ID=UNGHST
MSG91_OTP_TEMPLATE_ID=&lt;your template id&gt;

RESEND_API_KEY=&lt;your Resend key&gt;
RESEND_FROM=hello@unghost.com
SUPPORT_INBOX_EMAIL=support@unghost.com

PHONEPE_MERCHANT_ID=&lt;sandbox or live&gt;
PHONEPE_SALT_KEY=&lt;sandbox or live&gt;
PHONEPE_SALT_INDEX=1
PHONEPE_BASE_URL=https://api-preprod.phonepe.com/apis/pg-sandbox
# ALLOW_MOCK_PAYMENTS=true # uncomment only on staging if no live keys yet

PUSHER_APP_ID=...
PUSHER_KEY=...
PUSHER_SECRET=...
PUSHER_CLUSTER=ap2
```

```
INNGEST_EVENT_KEY=...
INNGEST_SIGNING_KEY=...

HMS_ACCESS_KEY=...
HMS_APP_SECRET=...
HMS_TEMPLATE_ID=...
HMS_REGION=in

R2_ACCOUNT_ID=...
R2_ACCESS_KEY_ID=...
R2_SECRET_ACCESS_KEY=...
R2_BUCKET=unghost-staging
R2_PUBLIC_BASE_URL=...

SENTRY_DSN=...
NEXT_PUBLIC_SENTRY_DSN=...
SENTRY_TRACES_SAMPLE_RATE=0.2
NEXT_PUBLIC_SENTRY_TRACES_SAMPLE_RATE=0.1

GOOGLE_CLIENT_ID=...
GOOGLE_CLIENT_SECRET=...
LINKEDIN_CLIENT_ID=...
LINKEDIN_CLIENT_SECRET=...

SLACK_WEBHOOK_ENGINEERING=...
SLACK_WEBHOOK_PROD=...
SLACK_WEBHOOK_INCIDENTS=...
```

Note. Repeat the same env block for the **unghost-prod** container with prod values. The only differences: NEXT_PUBLIC_APP_VERSION=v0.1.0 (or your git tag), all URLs change to https://www.unghost.com, Mongo URI points to prod cluster, Upstash points to prod DB, R2 bucket is unghost-prod, PhonePe is LIVE keys.

5. GitHub Actions secrets

Repo → **Settings** → **Secrets and variables** → **Actions** → **New repository secret**. Add each of the following.

CLOUDFLARE_API_TOKEN	Create at Cloudflare → My Profile → API Tokens → Create Token. Template: **Edit Cloudflare Workers**. Scope: account-level, Workers Scripts:Edit, Workers KV:Edit. Copy.
CLOUDFLARE_ACCOUNT_ID	Right sidebar of any Cloudflare dashboard page. 32-char hex.
CLOUDFLARE_REGISTRY_TOKEN	Cloudflare → Workers & Pages → Containers → Registry → Create token. Read+Write.
CLOUDFLARE_REGISTRY_USERNAME	From the same Registry page — usually your account ID or a generated username.
PROD_MIGRATOR_MONGODB_URI	The migrator Mongo URI you created in step 3.d. Do NOT use the app-rw user.
SENTRY_AUTH_TOKEN	From step 3.n — the auth token with project:releases scope.
SENTRY_ORG	Slug from step 3.n — e.g. `unghost`.
SENTRY_PROJECT	Slug from step 3.n — e.g. `unghost-server`.
SLACK_WEBHOOK_PROD	From step 3.r — the #prod webhook URL.

6. Database migration runbook

Migrations live in `server/db/migrations/*.js` and run via migrate-mongo. Each is idempotent — safe to re-run.

Staging

```
# from repo root, on your laptop
MONGODB_URI="&lt;staging URI&gt;" npm run migrate:up
MONGODB_URI="&lt;staging URI&gt;" npm run migrate:status
# expected: each migration listed as APPLIED
```

Production

Same commands, but use the **migrator** user URI (more privileges than app-rw) and only after merge to main. Best practice: do this through CI — the workflow already runs migrate:up on tag push using PROD_MIGRATOR_MONGODB_URI.

```
MONGODB_URI="&lt;prod migrator URI&gt;" npm run migrate:up
MONGODB_URI="&lt;prod migrator URI&gt;" npm run migrate:status
```

> **Warning.** Never run migrations from your laptop against prod unless CI is broken. Use the migrator user (not app-rw) so the audit trail in Atlas shows exactly which user made schema changes.

7. Seed data — do NOT seed production

`scripts/seed.ts` wipes every collection before re-seeding. It self-refuses if `NODE_ENV=production` or if the Mongo URI smells like prod (matches mongodb+srv with no *staging|dev|test* in the hostname).

When to seed

- Local dev — yes, after fresh `docker-compose up`.
- Staging — optional, if you want demo accounts for QA. Run once.
- Production — **NEVER**. Real users will lose data.

How to seed staging

```
MONGODB_URI="&lt;staging URI with 'staging' in hostname&gt;" npm run seed
```

How to escape the prod guard (last resort)

Only if you understand the data loss:

```
SEED_ALLOW_PROD=true MONGODB_URI="&lt;prod URI&gt;" npm run seed
# this will delete every collection in prod. Do not run.
```

8. First production deploy

1. All env vars set in Cloudflare Container (step 4) ✓
2. All GitHub Actions secrets set (step 5) ✓
3. Migrations applied to prod (step 6) ✓
4. DNS records pointing to containers (step 3.c, now resolvable) ✓
5. From the repo: `git tag -a v0.1.0 -m "first prod deploy" && git push --tags`.
6. Open the GitHub Actions tab — workflow **deploy-production** starts.
7. Workflow blocks at the **production** environment for approval — go to **Settings** → **Environments** → **production** and approve.
8. Watch the workflow continue: build, migrate, push image, deploy, smoke test.
9. When smoke passes (Slack #prod fires), open `https://www.unghost.com`.
10. Hit `https://www.unghost.com/api/health` — every adapter should show `mode: "live"`.

9. Post-deploy validation checklist

Run these in the order listed. Stop and fix anything that fails before moving on.

1. **Signup flow** — open `https://www.unghost.com/signup`, sign up with your real phone + email. Confirm the SMS OTP lands within 30 sec (MSG91 working).
2. **Email verify** — click the link in the verify email Resend sent. Land on success page.
3. **Resume upload** — drop a PDF on the dashboard. After parsing, check the R2 bucket — file should be there.
4. **Apply to a job** — apply to any active job. Atlas **Collections** → **applications** should show your new row.
5. **Pro upgrade** — click **Go Pro**. Complete payment with PhonePe sandbox card. Verify your `user.plan` in Atlas flips to `pro`.
6. **Cron firing** — in Inngest dashboard, you should see `sla-sweep` and `subscription-sweep` registered. Trigger manually to confirm.
7. **Sentry** — visit `/api/_test_error` (if you've created one) or trigger any 500. Confirm the event lands in Sentry.
8. **Better Stack** — uptime monitor should show GREEN. Status page should show **All systems operational**.
9. **Pusher** — send a chat message from the recruiter side. Student side should receive it in real-time without page refresh.
10. **100ms** — start a coaching session as instructor. Student should be able to join.

10. Monitoring + alerting

Sentry alert rules

1. **Alerts** → **Create Alert**.
2. **Issue alert**: **When a new issue is created** → **Send notification to Slack** → channel `#incidents`.
3. **Metric alert**: **When error rate exceeds 1% over 5 minutes** → Slack #incidents + email.
4. **Metric alert**: **When p95 latency exceeds 1500ms over 5 minutes** → Slack #engineering.

Better Stack alert rules

1. Monitor **www.unghost.com** down for 60 seconds → Slack #incidents, SMS to founder.
2. Monitor **/api/health** returns non-200 → same.

Atlas alerts

1. Project → **Alerts** → Create. Connection count > 80% of max → email + Slack via Atlas webhook.
2. Replication lag > 5s → email.
3. Disk usage > 75% → email.

11. Rollback procedure

Container rollback (most common)

```
npx wrangler containers rollback unghost-prod
# rolls back to the previous successful image tag
# health check should pass again within 30 sec
```

DB rollback (rare — only if a migration broke prod)

```
MONGODB_URI="&lt;prod migrator URI&gt;" npm run migrate:down
# rolls back the latest migration only
```

> **Warning.** Most DB migrations are NOT safely reversible at runtime. Adding a column? Down migration drops it but you lose data. Renaming? Down loses the new column data. Prefer to **roll forward** with a fix migration when possible.

Post-mortem template

Create a doc within 24h of any incident with: timeline, root cause, customer impact, what we tried, what worked, action items. Share in #engineering.

12. Day-one ops checklist

First 24 hours after going live, watch these in order of priority.

1. Sentry dashboard — every red error gets a triage in #engineering.
2. Better Stack — make sure /api/health stays green.
3. Atlas → Metrics → connection count, slow queries. Spike means index missing.
4. Cloudflare → Analytics → bandwidth, requests, threats. A 522 spike means container CPU exhausted.
5. Upstash → Usage. If close to 10k commands/day, upgrade to Pay-as-you-go before midnight (free tier resets).
6. PhonePe dashboard → transactions. Any failed payments → investigate.
7. Anthropic console → token usage. If burning faster than expected, look for runaway loops.
8. Slack #prod and #incidents — keep them open all day.

13. Cost tracking + spend caps

Cloudflare	Free DNS + Workers + R2 (10GB) + Containers free tier covers ~100k req/day. Set up alerts at Account → Billing → Usage alerts.
Atlas	M0 free indefinitely for staging. M10 prod ~USD 60/mo. **Billing** → **Alerts** → set USD threshold.
Upstash	Free tier 10k cmds/day. Set monthly cap under Account → Usage → Budget.
Anthropic	Pay-as-you-go. **Settings** → **Billing** → **Usage limits** → monthly cap (recommend USD 100 start).
MSG91	Prepaid. Recharge wallet when low. ~INR 0.15/SMS for transactional.
Resend	Free 3k emails/mo. After that USD 20/mo for 50k.
PhonePe	1.99% per successful transaction, no monthly fee. Reconcile in their dashboard.
Pusher	Free 200k msgs/day. Sandbox tier covers most early use.
Inngest	Free 50k function runs/mo.
100ms	Free 10k minutes/mo.
Sentry	Free 5k events/mo. Team plan ~USD 26/mo for 50k events.
Better Stack	Free 10 monitors. Paid tier USD 25/mo for more + on-call.

Set ALL spend caps before going live. A bug in a rate limiter could empty your Anthropic credits in hours.

14. Troubleshooting appendix

PhonePe X-VERIFY signature mismatch

Cause: salt key or salt index wrong, or you're signing the wrong path. The signature is: `SHA256(base64payload + endpoint_path + saltKey) + ### + saltIndex`. Make sure endpoint_path includes the leading slash (e.g. `/pg/v1/pay`). Check that your SALT_INDEX matches what PhonePe issued (usually 1).

MSG91 returns 'Invalid Auth Key'

Cause: the auth key is correct but the DLT-approved template hasn't propagated to MSG91 yet. Check the MSG91 console under Templates — should show "Approved". If still pending after 48 hours of DLT approval, raise a ticket with MSG91 support.

Resend 'domain not verified'

Cause: DNS hasn't propagated, or you accidentally proxied a CNAME through Cloudflare. Resend records must be DNS only (grey cloud, not orange). Wait 15-30 min, then click Verify again. Confirm with `dig +short TXT resend._domainkey.unghost.com`.

Atlas connection refused

Cause: your IP isn't on the access list, or you're using `mongodb://` instead of `mongodb+srv://` with Atlas. Add 0.0.0.0/0 temporarily to confirm; if that works, you have an allowlist issue. If still failing, check DNS: `nslookup -type=SRV _mongodb._tcp.unghost-prod.xxx.mongodb.net`.

Cloudflare 522 / 525

Cause: container's health check is failing, or container is down. Run `wrangler containers logs unghost-prod --tail` to see what the container is doing. Common reasons: out of memory (bump to 2GB), Mongo connection refused (IP allowlist), missing env var causing startup crash.

Sentry isn't receiving events

Cause: `SENTRY_DSN` not set in container env, OR your config has `process.env.NODE_ENV !== 'production'` gating it (we set that in code — verify NODE_ENV is set to production in the container).

OAuth callback URL mismatch

Cause: the redirect URI in Google Cloud Console doesn't exactly match what NextAuth is computing. NextAuth uses NEXTAUTH_URL + `/api/auth/callback/google`. Make sure NEXTAUTH_URL is the exact prod URL (https + www) and that this exact URI is whitelisted in Google.

Cron jobs not firing

Cause: Cloudflare Containers doesn't support Vercel-style cron triggers natively. Use Inngest (already wired) or set up a separate Worker Cron Trigger that hits `https://www.unghost.com/api/cron/sla-sweep` with the `Authorization: Bearer $CRON_SECRET` header.

15. Glossary

DLT	Distributed Ledger Technology — TRAI mandate for all commercial SMS in India. You register your entity, sender ID (header), and SMS templates on a telecom operator's DLT portal before MSG91 (or any gateway) will actually deliver SMS.
KYC	Know Your Customer — identity verification step required by payment gateways and SMS providers in India.
GST	Goods and Services Tax — India's unified sales tax. GSTIN (15-char ID) is required for B2B payments and PhonePe merchant onboarding.
PAN	Permanent Account Number — Indian tax ID. Personal (individual) or company (firm-level).
PITR	Point-in-time Recovery — Atlas backup feature letting you restore the DB to any second within the retention window (7 days default).
SLA	Service Level Agreement — in our domain, refers to the time-to-grade promise recruiters make on candidate applications.
RSC	React Server Component — Next.js 14 default. Renders on the server, sends serialised HTML/payload, no client bundle for that component.
X-VERIFY	PhonePe's signature header on every request. SHA256 of base64 payload + endpoint path + salt key, suffixed with ### + salt index.
PITR	Point-in-time Recovery — Atlas backup.
CSP	Content Security Policy — HTTP header that tells the browser what scripts/styles/images are allowed to load.
DPDP	Digital Personal Data Protection Act — India's GDPR-equivalent. Requires consent management and right-to-erasure.
Webhook	An HTTP POST your server receives when something happens in an external service (payment success, SMS delivered, etc).
Wrangler	Cloudflare's CLI for managing Workers, Containers, R2, etc.
Inngest	Background job and workflow runner. Reliable, retryable, durable.
Pusher	Hosted WebSocket service for real-time push to browsers.
R2	Cloudflare's S3-compatible object storage. Free egress.
Sentry	Error monitoring SaaS. Captures stack traces from server + client.
Better Stack	Uptime monitoring + status page service.

End of guide. If you got here and everything in section 9 is green: congrats, you're live.